Smart Healthcare Management

Data Management and File Structures 1 Homework Report

Prepared By:
Riza Can Tay

Submitted To:
Prof. Dr. İbrahim Emiroglu

Yildiz Technical University - Department of Mathematical Engineering

May 24, 2026

Contents

1 Requirements Analysis

This project, an assignment for the MTM2522 Data Management and File Structures 1 course in the Department of Mathematical Engineering at Yildiz Technical University, is a central database system designed to digitize some basic daily operations of a large hospital network consisting of multiple hospitals. These operations include outpatient appointments, simple doctor-patient interactions, patient records, personnel management, appointment tracking, and financial billing. Pharmacy inventory tracking, hospital inpatient wards (operating rooms, etc.), and many other modules are excluded from the conceptual scope of this project.

1.1 Entities

Below is a list of the entities that form the basis of the system, their attributes, and key structures (PK, FK, UNIQUE):

- **Person (Superclass):** Contains basic information of all persons registered.
  - `Person_ID` (PK): Unique primary key automatically assigned to each individual.
  - `National_ID` (UNIQUE): Person's official national identification number.
  - `Name` (Composite Attribute): Person's name information.
    - `First_Name`: Person's first name information.
    - `Last_Name`: Person's last name information.
  - `Date_of_Birth`: Person's date of birth.
  - `Gender`, `Blood_Type`: Person's gender and blood type information.
  - `Phone_Number`: Person's contact number.
  - `Address` (Composite Attribute): Person's physical address information.
    - `Country`: Country of residence.
    - `City`: City of residence.
    - `District`: District or borough information.
    - `Street`: Street name of the address.
    - `Building_No`: Building number of the residence.
    - `Apartment_No`: Apartment number of the residence.
    - `Zip_Code`: Postal/ZIP code of the address.
  - `Email` (UNIQUE), `Password_Hash`: Unique email address and encrypted password are used for secure system login.
- **Medical_Staff (Intermediate Superclass):** Groups the personnel authorized to perform medical interventions and inherits from `Person`.
  - `Person_ID` (PK): Primary key that references the `Person` table.
  - `Department_ID` : Reference from the department the employee belongs to.
- **Patient (Subclass):** Represents individuals receiving medical services from the hospital and inherits from `Person`.

- `Person_ID` (PK): Primary key that references the `Person` table.
- `Allergies` (Multivalued): Reactions by patient's immune system to substances.

– **Doctor (Medical Subclass):** Represents doctors authorized to provide medical diagnosis and treatment and inherits from `Medical_Staff`.

- `Person_ID` (PK): Primary key that references the `Person` table.
- `Specialty`: Doctor's area of expertise.
- `License_Number` (UNIQUE): Doctor's official license number.
- `Treatments` (Multivalued): Medical methods that doctor has been trained to use to cure patient.

– **Nurse (Medical Subclass):** Represents nurses working in the hospital and inherits from `Medical_Staff`.

- `Person_ID` (PK): Primary key that references the `Person` table.

– **Admin (Subclass):** Represents the senior staff responsible for the administration and inherits from `Person`.

- `Person_ID` (PK): Primary key that references the `Person` table.
- `Role_Level`: Administrator's level of authority.
- `MFA_Secret` (NOT NULL): Security key used for two-factor authentication.

– **Staff (Subclass):** Represents non-medical support staff (cleaning, security, consulting, etc.) and inherits from `Person`.

- `Person_ID` (PK): Primary key that references the `Person` table.
- `Job_Title`: Job description of the staff.

– **Disease:** Refers to process of identifying a disease or condition

- `Disease_ID` (PK): Unique registration number assigned to this disease by system.
- `Disease_Name`: General name of disease.
- `Disease_Type`: General classification of disease (e.g., Chronic, Acute, Infectious).
- `Symptoms (Multivalued)`: Set of symptoms associated with this disease (e.g., shortness of breath, high fever).

– **Hospital:** Defines physical and organizational data of hospital buildings.

- `Hospital_ID` (PK): Hospital's unique code.
- `Hospital_Name`: Official name of hospital.
- `Address` (Composite Attribute): Hospital's address information.
  - `Country`: Country where hospital is located.
  - `City`: City where hospital is located.

    - `District`: District or borough of hospital.
    - `Street`: Street name of hospital address.
    - `Building_No`: Building number of hospital.
    - `Zip_Code`: Postal/ZIP code of hospital address.
  - `Contact_Number`: Contact number of hospital.

- **Department:** Defines medical or administrative departments operating within hospital.
  - `Department_ID` (PK): Unique registration number of department.
  - `Hospital_ID` : Hospital's unique code.
  - `Department_Name`: Name of the department (Cardiology, Neurology, etc.).
  - `Extension_Number`: Internal phone number that provides direct access to department.

- **Room (Weak Entity):** Defines physical rooms within hospital departments and is directly dependent on `Department` to exist.
  - `Department_ID` (PK): Unique registration number of department.
  - `Room_Number` (Partial Key): Door number within department and combines with `Department_ID` to form a composite primary key.
  - `Room_Type`: Purpose of room (Examination Room, Laboratory, Polyclinic, etc.).

- **Appointment (Associative Entity):** Central process table that tracks relationship between doctor and patient.
  - `Appointment_ID` (PK): Unique registration number of appointment.
  - `Patient_ID`* , `Doctor_ID`* : References identifying parties to appointment.
  - `Appointment_Date`, `Appointment_Time`: Date and time of appointment.
  - `Status_ID`: Current status of appointment (0: Cancelled 1:Scheduled 2: Completed, etc.).
  - `Chief_Complaint`: Primary reason for the patient's visit.

- **Prescription (Weak Entity):** Records prescriptions written for pations as a result of completed appointments.
  - `Appointment_ID` : Unique registration number of appointment.
  - `Medication_Name`: Name of prescribed medication.
  - `Dosage`: Dosage of medication to be used.
  - `Instructions`: Doctor's instructions regarding the use of medication.

- **Lab_Result:** Records medical tests and their results requested during appointments.
  - `Lab_ID` (PK): The unique record number of test result.
  - `Appointment_ID` : Unique registration number of appointment.

- `Test_Type`: Type of test performed (Completed Blood Count, MRI, X-ray, etc.).
- `Result_Value`: Numerical or textual result of laboratory
- `Reference_Range`: Reference range considered healthy for relevant test.
- `Test_Date`: Date the test was performed.

– **Insurance:** Tracks the private or state-supported health insurance policies held by patients.

- `Insurance_ID` (PK): Unique number of insurance record.
- `Patient_ID`* : Reference of patient to whom the policy belongs.
- `Provider_ID` : Reference of insurance provider.
- `Policy_Number`: Policy number assigned by insurance provider.
- `Coverage_Rate`: Percentage of insurance's coverage of invoice amount (e.g., %80).

– **Billing:** Tracks financial payment status resulting from completed services.

- `Bill_ID` (PK): Unique registration number of invoice.
- `Appointment_ID` : Unique registration number of the appointment.
- `Total_Amount`: Total cost of procedures performed.
- `Patient_Share`: Net amount the patient must pay after insurance discount.
- `Payment_Status`: Current payment status of invoice (0: Pending, 1: Paid, etc.).

– **Insurance_Provider:** Represent private insurance companies or government institutions that patient has agreements with.

- `Provider_ID` (PK): Unique number assigned to provider by system.
- `Provider_Name`: Official name of insurance provider (Allianz, AXA, etc.).
- `Tax_Number`: Provider's official tax number.
- `Provider_Email`: Provider's official contact email.

– **Payer (Union):** Representing anyone who can make payment, formed by union of `Patient` and `Insurance_Provider`.

- `Payer_ID` (PK): Unique number assigned to provider by system.
- `Payer_Type`: Discriminator that indicates whether payer is natural person or legal entity.

– **Payment:** Tracks individual financial transactions made to settle billing invoice.

- `Payment_ID` (PK): Unique registration number of transaction receipt.
- `Bill_ID` : Reference to the invoice being paid.
- `Payer_ID` : Reference to entity who made payment.
- `Amount_Paid`: Exact monetary value transferred in transaction.

- `Payment_Date`: Date and time the transaction was completed.

** **Note:** `Patient_ID` and `Doctor_ID` external keys are named aliases to highlight logic and readability. In the physical database model, these fields directly reference `Person_ID` fields of the corresponding subclasses.*

1.2 Relationships

The relationships and cardinalitys of the entities are as follows:

Hierarchical Relationships / Inheritance (IS-A)

- **Person – Patient / Admin / Staff:** All system users are inherits from `Person` entity.
- **Person – Medical_Staff – Doctor / Nurse:** Medical personnel are structured through a multi-level inheritance hierarchy to ensure proper attribute delegation and specialization.

Functional Relationships (HAS-A)

- **Hospital – Department (1:N) :** A single hospital structure contains multiple departments. Each department belongs strictly to this main hospital entity.
- **Department – Medical_Staff (1:N) :** A department employs multiple medical staff members. To maintain operational integrity, each medical staff member is officially registered to only one primary department.
- **Department – Room (1:N) :** A department may have multiple rooms, but each room belongs to its own department.
- **Patient – Appointment (1:N) :** A registered patient can book multiple appointments over time, but each specific appointment record is assigned to exactly one patient.
- **Doctor – Appointment (1:N) :** A doctor can conduct multiple appointments, but each individual medical encounter is managed by exactly one specific doctor.
- **Appointment – Prescription (1:1) :** An appointment can generate at most one prescription, which is directly tied to that specific encounter.
- **Appointment – Lab_Result (1:N) :** A single appointment can result in multiple laboratory tests or results, all of which are tracked back to the originating encounter.
- **Appointment – Billing (1:1) :** Each appointment generates exactly one primary billing record for financial tracking.
- **Patient – Insurance (1:1) :** A patient holds exactly one primary active insurance policy within the system.
- **Insurance_Provider – Insurance (1:N) :** A single insurance provider company can issue multiple insurance policies to different patients, but each policy is strictly managed by one provider.
- **Billing – Payment (1:N) :** A single generated billing invoice can receive multiple separate payments until total amount is settled.
- **Payer – Payment (1:N) :** A payer entity can make multiple financial payments over time, but each payment transaction is traced back to exactly one payer.
- **Patient – Doctor (M:N) :** A patient can be treated by multiple doctors and a doctor can treat multiple patients.
- **Patient – Disease (M:N) :** A patient can be diagnosed with multiple diseases and a single disease can be diagnosed in multiple patients. Associative relationship (`Diagnosed_With`) captures specific attributes.

1.3 Constraints

To preserve data integrity and accurately reflect real-world hospital operations within the database, the following constraints have been defined:

Unique Constraints

Rules enforcing uniqueness to prevent duplicate records and identity conflicts across the database.

- **Citizen and Contact Uniqueness:** National identification number (`National_ID`) and email address (`Email`) in `Person` entity must be globally unique. Single individual cannot be registered multiple times.
- **Professional Identity Uniqueness:** Professional license or registration number (`License_Number`) within `Doctor` entity must be strictly unique.

Domain & Check Constraints

Rules ensuring that inputted data strictly adheres to logical boundaries and physical realities.

- **Date of Birth Logic:** `Date_of_Birth` of a `Person` cannot exceed current system date (`CURRENT_DATE`). Furthermore, it must strictly precede any scheduled appointment, laboratory test dates, from the time the payment was received, the time of diagnosis.
- **Retrospective Appointment Restriction:** `Appointment_Date` in `Appointment` entity cannot be set to a date prior to the exact moment of booking.
- **Positive Values:** `Total_Amount` and `Patient_Share` in `Billing` and `Amount_Paid` in `Payment` must never hold negative values (`> 0` or `>= 0`).
- **Share Limit:** A patient's financial responsibility cannot exceed the total cost of the invoice ($\texttt{Total\_Amount} \geq \texttt{Patient\_Share}$).
- **Overpayment Prevention:** Sum of all `Amount_Paid` values in `Payment` entity associated with a specific `Bill_ID` must be less than or equal to the `Total_Amount` of that originating bill. ($\texttt{Total\_Amount} \geq \sum \texttt{Amount\_Paid}$)
- **Insurance Coverage Limits:** `Coverage_Rate` in the `Insurance` table must always be a percentage value between 0 and 100 ($0 \leq x \leq 100$).

Business Logic & Structural Constraints

The enforcement of the hospital's operational workflows through referential integrity.

- **Clinical Assignment Rule:** By hierarchical design, each medical professional (Doctor or Nurse) in the `Medical_Staff` superclass must be assigned to exactly one department (`Department_ID`) via a mandatory 1:N relationship.
- **Double-Booking Prevention:** A specific doctor (`Doctor_ID`) cannot be scheduled for multiple patients on the same `Appointment_Date` and `Appointment_Time`. Similarly, a patient (`Patient_ID`) cannot book multiple appointments at the exact same date and time.
- **Communication Line Separation:** Institutional contact number (`Contact_Number`) of a `Hospital` cannot be identical to the personal phone numbers (`Phone_Number`) of individuals (`Person`) registered in the system. Corporate lines must be strictly isolated.
- **Process Dependency:** The generation of a `Prescription` and a `Billing` invoice can only occur for appointments where the `Status_ID` equals 2 (Completed). Scheduled (1) or Canceled (0) appointments cannot yield any medical or financial records.
- **Security Mandate:** Users assigned to the `Admin` subclass are strictly required to possess a Multi-Factor Authentication secret key (`MFA_Secret`) to access the system (`NOT NULL`).

2 Diagrams

2.1 Specialization/Generalization Hierarchy

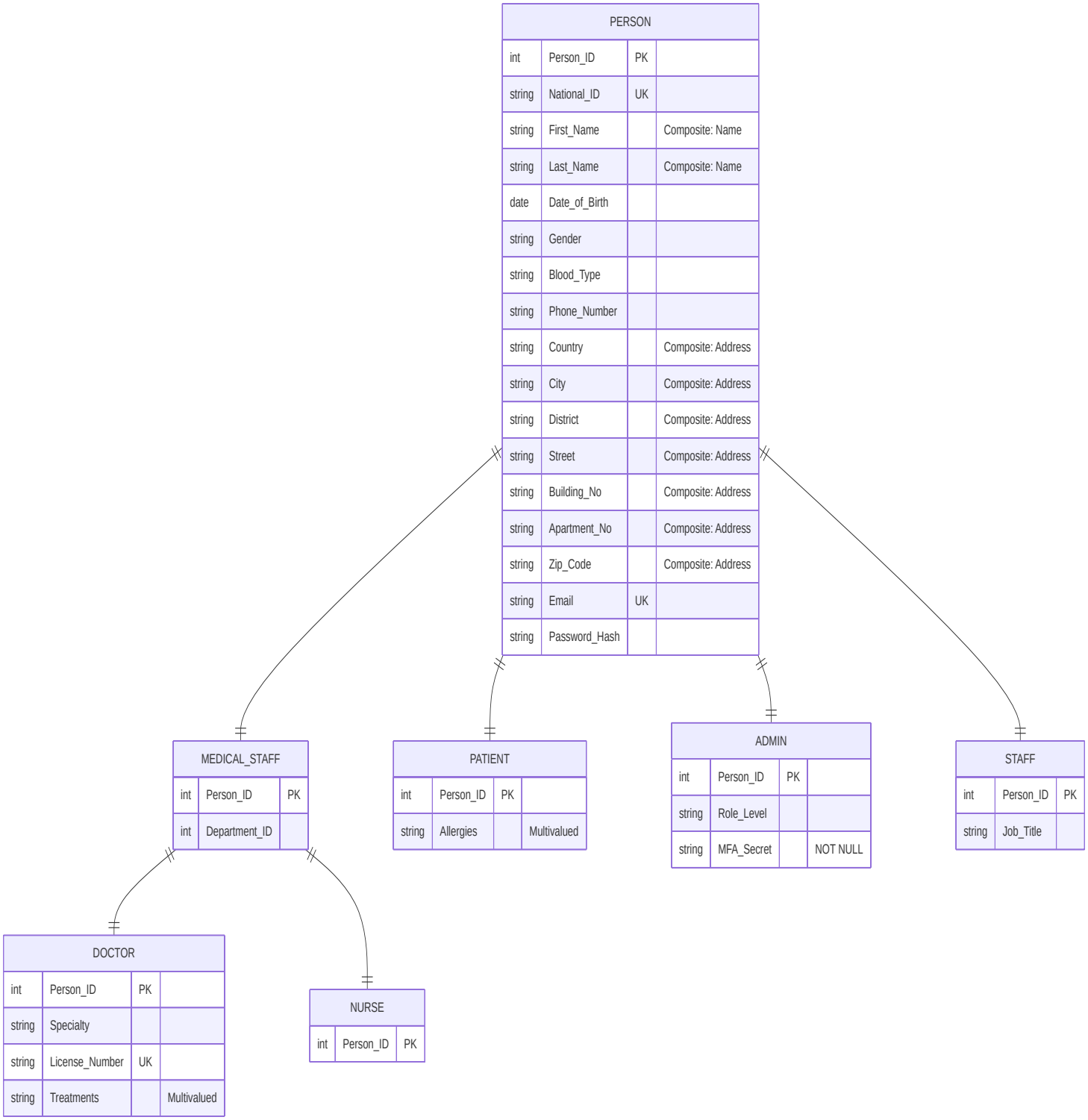

Figure 1: Specialization/Generalization Hierarchy

2.2 Conceptual Data Model

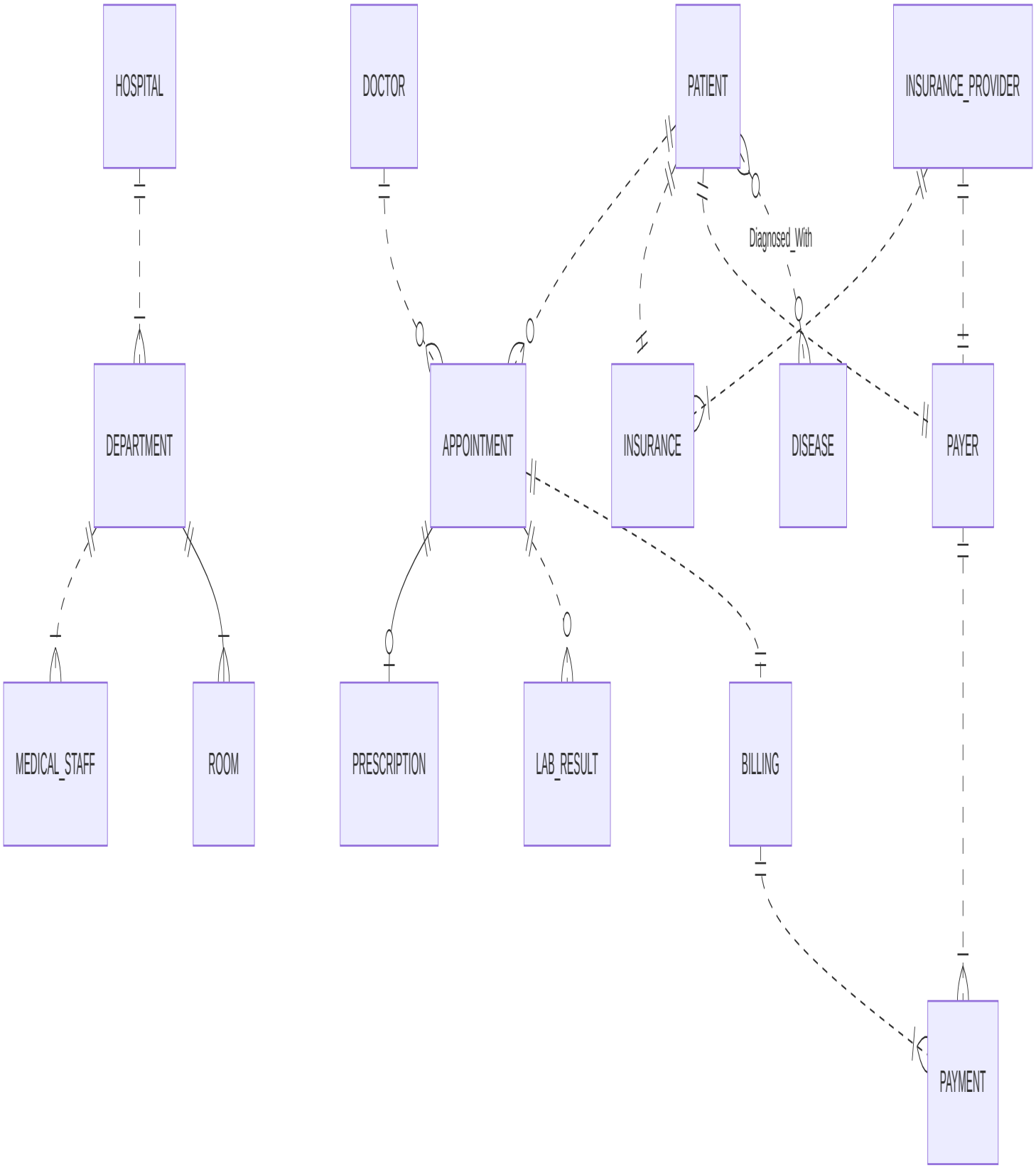

Figure 2: Conceptual Data Model

2.3 EER Diagram

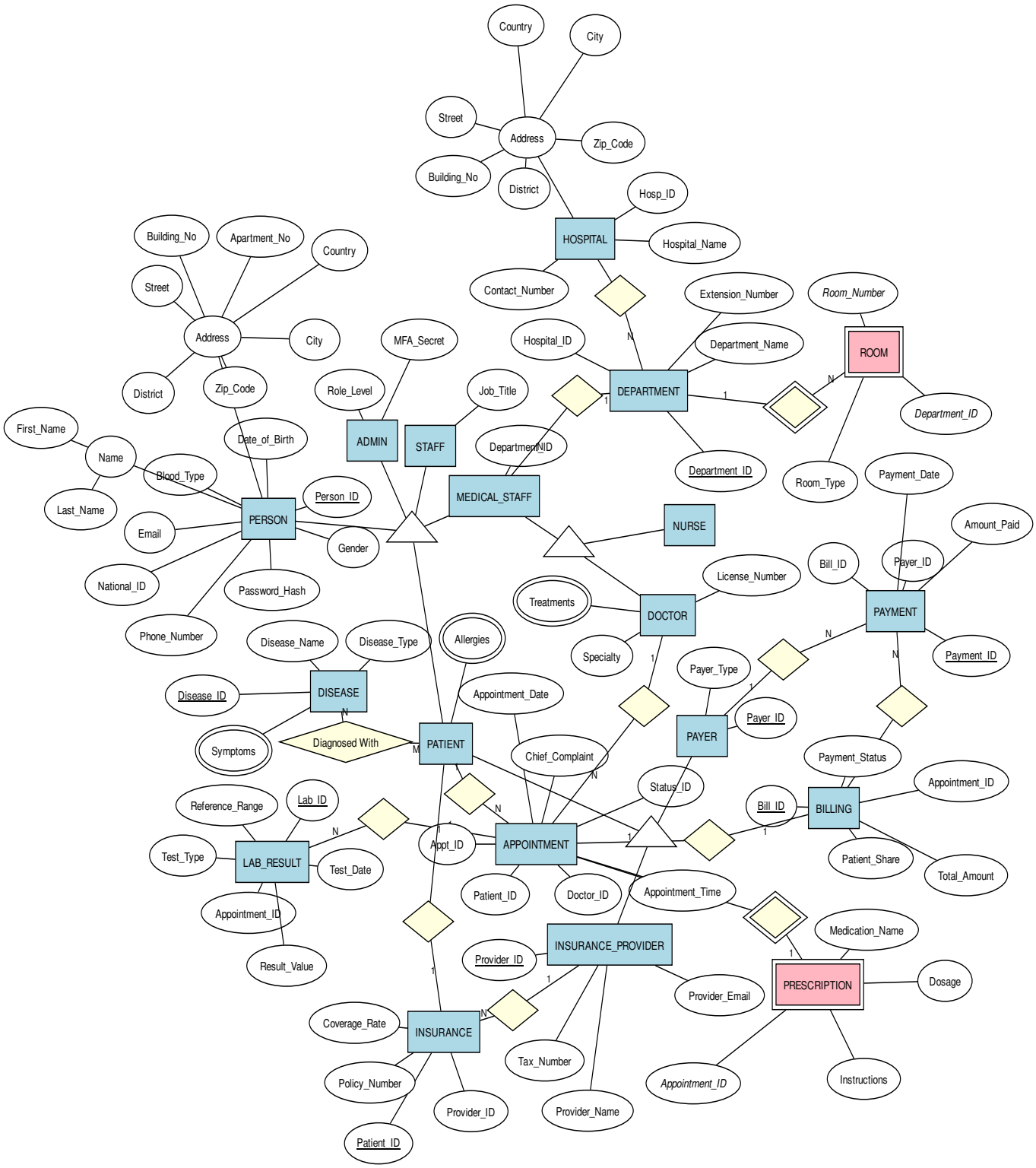

Figure 3: EER Diagram

3 Mapping to Relational Model

In this phase, the conceptual Extended Entity-Relationship (EER) model is logically mapped into a relational database schema. The mapping strictly adheres to relational model constraints, specifically achieving First Normal Form (1NF) by eliminating multivalued attributes and composite attributes.

3.1 Standard Entities and Hierarchical Mapping

To maintain relational integrity and a clean architectural overview, all standard entities and hierarchical inheritance structures are mapped below by focusing strictly on their structural bones: Primary Keys (PK) and Foreign Keys (FK). All non-key descriptive attributes are omitted in this view to highlight the underlying relational connectivity.

- **PERSON**
  - `Person_ID` : *Primary Key*
- **MEDICAL_STAFF**
  - `Person_ID` : *Primary Key, Foreign Key*
  - `Department_ID` : *Foreign Key*
- **PATIENT**
  - `Person_ID` : *Primary Key, Foreign Key*
- **DOCTOR**
  - `Person_ID` : *Primary Key, Foreign Key*
  - `License_Number`$^{\text{UNIQUE}}$
- **NURSE**
  - `Person_ID` : *Primary Key, Foreign Key*
- **ADMIN**
  - `Person_ID` : *Primary Key, Foreign Key*
- **STAFF**
  - `Person_ID` : *Primary Key, Foreign Key*
- **HOSPITAL**
  - `Hospital_ID` : *Primary Key*
- **DEPARTMENT**
  - `Department_ID` : *Primary Key*
  - `Hospital_ID` : *Foreign Key*
- **DISEASE**
  - `Disease_ID` : *Primary Key*
- **LAB_RESULT**
  - `Lab_ID` : *Primary Key*
  - `Appointment_ID` : *Foreign Key*
- **INSURANCE_PROVIDER**
  - `Provider_ID` : *Primary Key*

- **INSURANCE**
  - `Patient_ID` : *Primary Key, Foreign Key*
  - `Provider_ID` : *Foreign Key*
- **PAYER**
  - `Payer_ID` : *Primary Key*
- **BILLING**
  - `Bill_ID` : *Primary Key*
  - `Appointment_ID` : *Foreign Key*
- **PAYMENT**
  - `Payment_ID` : *Primary Key*
  - `Bill_ID` : *Foreign Key*
  - `Payer_ID` : *Foreign Key*

3.2 Handling Multivalued Attributes

To comply with 1NF, multivalued attributes (`Allergies`, `Treatments`, `Symptoms`) are extracted from their parent entities and mapped into new, independent relations. The primary key of these new relations is a composite of the parent's primary key and the attribute value itself.

- **PATIENT_ALLERGIES**
  - `Person_ID` : *Primary Key, Foreign Key*
  - `Allergy_Name` : *Primary Key*
- **DOCTOR_TREATMENTS**
  - `Person_ID` : *Primary Key, Foreign Key*
  - `Treatment_Name` : *Primary Key*
- **DISEASE_SYMPTOMS**
  - `Disease_ID` : *Primary Key, Foreign Key*
  - `Symptom_Name` : *Primary Key*

3.3 Mapping Weak Entities

Weak entities cannot exist independently. They are mapped by absorbing the primary key of their owning entity as a foreign key, which then acts as part of their composite primary key (or sole primary key in 1:1 identifying relationships).

- **ROOM**
  - `Department_ID` : *Primary Key, Foreign Key*
  - `Room_Number` : *Primary Key*
  - `Room_Type`
- **PRESCRIPTION**
  - `Appointment_ID` : *Primary Key, Foreign Key*
  - `Medication_Name`
  - `Dosage`
  - `Instructions`

3.4 Resolving Many-to-Many (M:N) Relationships

M:N relationships are resolved by creating associative (junction) tables that contain foreign keys referencing the primary keys of the participating entities.

- **APPOINTMENT**
  - `Appointment_ID` : *Primary Key*
  - `Patient_ID` : *Foreign Key*
  - `Doctor_ID` : *Foreign Key*
  - `Appointment_Date`
  - `Appointment_Time`
  - `Status_ID`
  - `Chief_Complaint`

  (Resolves the implicit M:N relationship between Patient and Doctor.)
- **DIAGNOSED_WITH**
  - `Patient_ID` : *Primary Key, Foreign Key*
  - `Disease_ID` : *Primary Key, Foreign Key*
  - `Diagnosis_Date`
  - `Diagnosed_Status`

  (Resolves the M:N relationship between Patient and Disease.)

3.5 Relational Database Schema

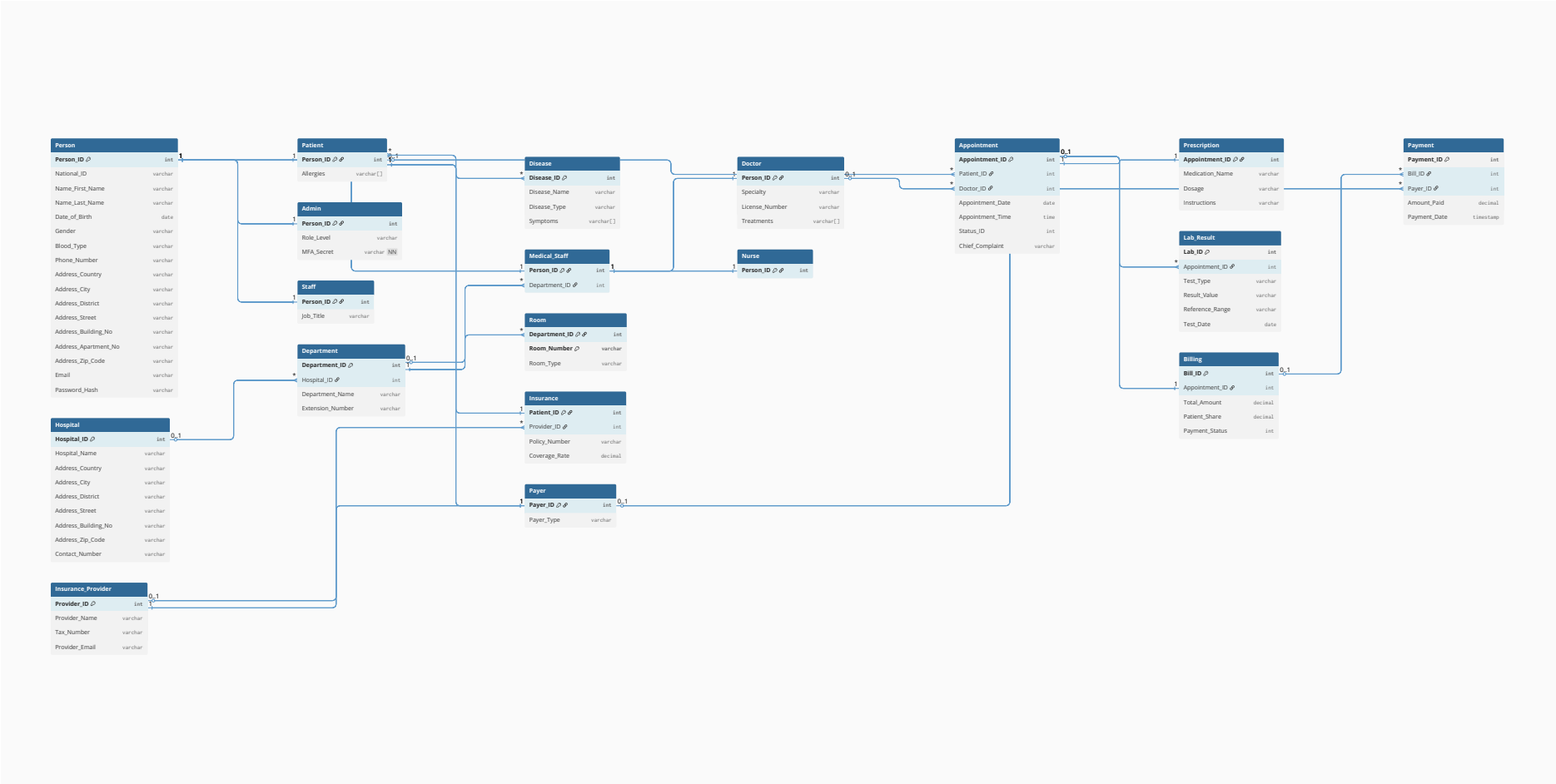

Figure 4: Relational Database Schema

4 Functional Dependencies

In this section, functional dependencies (FDs) are identified for three foundational tables that represent the biological, physical, and operational roots of the system: **PERSON**, **HOSPITAL**, and **APPOINTMENT**.

1. PERSON Table

`PERSON` table is the hierarchical root of all individuals in the system. `Person_ID` is the surrogate primary key and strictly determines all biological and contact attributes. Furthermore, since `National_ID` (Citizen ID) and `Email` are enforced as globally unique constraints, they serve as *Candidate Keys*. Therefore, they can also uniquely determine all other attributes in the relation independently.

- `Person_ID` $\rightarrow$ `National_ID`, `First_Name`, `Last_Name`, `Date_of_Birth`, `Gender`, `Blood_Type`, `Email`, ...
- `National_ID` $\rightarrow$ `Person_ID`, `First_Name`, `Last_Name`, `Email`, ...
- `Email` $\rightarrow$ `Person_ID`, `National_ID`, `First_Name`, ...

2. HOSPITAL Table

`HOSPITAL` table defines the physical healthcare facilities. The primary key `Hospital_ID` functionally determines the name, contact details, and the full composite address of the facility. A hospital's ID is completely sufficient to derive its exact physical location.

- `Hospital_ID` $\rightarrow$ `Hospital_Name`, `Contact_Number`, `Country`, `City`, `District`, `Street`, `Building_No`, `Zip_Code`

3. APPOINTMENT Table

The `APPOINTMENT` table is the operational core connecting patients and doctors. While the primary key `Appointment_ID` determines all operational details of a session, there is a strict operational business rule: *A specific doctor cannot have two different appointments at the exact same date and time.* Thus, the composite of (`Doctor_ID`, `Appointment_Date`, `Appointment_Time`) acts as a powerful candidate key that functionally determines the exact appointment, including who the patient is and what the complaint is.

- `Appointment_ID` $\rightarrow$ `Patient_ID`, `Doctor_ID`, `Appointment_Date`, `Appointment_Time`, `Status_ID`, `Chief_Complair`
- `Doctor_ID`, `Appointment_Date`, `Appointment_Time` $\rightarrow$ `Appointment_ID`, `Patient_ID`, `Status_ID`, `Chief_Complair`

5 Normalization

Normalization is the formal mathematical process of analyzing relational schemas based on their functional dependencies (FDs) and primary keys to minimize data redundancy and eliminate insertion, deletion, and update anomalies.

5.1 First Normal Form (1NF)

A relation R is in 1NF if all attributes contain only atomic values. This implies that each cell in the table must contain a single, indivisible value from the attribute's domain, effectively forbidding multi-valued attributes or nested relations.

The existing system satisfies 1NF as every attribute in all relations contains only atomic values.

- **Atomicity:** Attributes such as *Patient_ Name*, *Doctor_ Specialty*, and *Room_ Number* hold single values for each tuple.
- **Elimination of Multi-valued Attributes:** Multi-valued attributes like *Allergies* or *Symptoms* are not stored as arrays or comma-separated strings within the `PATIENT` or `APPOINTMENT` tables. Instead, they are decomposed into separate weak entities or associative tables (e.g., `PATIENT_ALLERGY`), ensuring that $\forall t \in R, t[A]$ is atomic.

If we stored patient allergies directly within the `PATIENT` relation as a multi-valued attribute.

To satisfy 1NF, the multi-valued attribute is decomposed into a separate weak entity, ensuring domain atomicity.

Patient_ID	Name	BirthDate	Allergies (Violation)
P101	John Doe	1985-04-12	Penicillin, Peanuts
P102	Jane Smith	1992-08-25	Sulfur, Dust

Table 1: Non-1NF Table: Multi-valued attributes cause search anomalies.

Patient_ID (FK)	Allergy_Name
P101	Penicillin
P101	Peanuts
P102	Sulfur
P102	Dust

Table 2: 1NF Compliant: Decomposed `PATIENT_ALLERGY` relation.

5.2 Second Normal Form (2NF)

A relation R is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key. This prevents partial functional dependencies, where an attribute depends only on a subset of a composite primary key rather than the key in its entirety.

A relation violates 2NF only if it has a composite primary key and a partial dependency exists.

- **Single-Attribute Keys:** Most critical tables (e.g., `PATIENT`, `DOCTOR`, `DEPARTMENT`) utilize a single-attribute primary key (Surrogate Keys like *Patient_ID*). Since a proper subset of a single-attribute key is an empty set, partial functional dependency is mathematically impossible.
- **Composite Keys:** In relations with composite keys, such as `APPOINTMENT (Doctor_ID, Start_Time)`, all non-prime attributes like *Diagnosis* or *Status* are fully functionally dependent on the **entire** key. Knowledge of only *Doctor_ID* is insufficient to determine a specific *Diagnosis* without the temporal context of *Start_Time*.

In our schema, we utilize a composite primary key to uniquely identify rooms, combining the Department and the Room itself: *(Department_ID, Room_Number)*. If we had incorrectly included the department's name within this room-specific relation, a 2NF violation would occur.

Department_ID	Room_Number	Department_Name (Violation)
DEP-1	101	Cardiology
DEP-1	102	Cardiology
DEP-2	101	Neurology

Table 3: Non-2NF Table: *Department_Name* is partially dependent only on *Department_ID*.

To strictly adhere to 2NF, our actual design eliminates this partial dependency. The *Department_Name* is exclusively stored in the `DEPARTMENT` table. In our actual `ROOM` table, attributes such as *Capacity* are fully functionally dependent on the **entire** composite key *(Department_ID, Room_Number)*, preventing massive data redundancy.

5.3 Third Normal Form (3NF)

A relation schema R is in third normal form with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \to \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

– $\alpha \to \beta$ is a trivial functional dependency.

Department_ID (PK/FK)	Room_Number (PK)
DEP-1	101
DEP-1	102
DEP-2	101

Table 4: 2NF Compliant: The actual `ROOM` relation without partial dependencies.

– α is a superkey for R.

– Each attribute A in $\beta - \alpha$ is contained in a candidate key for R.

The schema eliminates transitive dependencies by ensuring that non-prime attributes are determined only by the primary key.

- **Example:** In the `DOCTOR` relation, we store *Department_ID* but not *Department_Name* or *Department_Location*. If we had included *Department_Name*, a transitive dependency would exist: $Doctor\_ID \rightarrow Department\_ID \rightarrow Department\_Name$.
- By isolating Department details into a separate `DEPARTMENT` table, we ensure that for every $X \rightarrow A$, X is a superkey, satisfying the 3NF requirements across the entire relational forest.

If the `DOCTOR` table stored both the Department ID and the physical location of that department.

Doctor_ID	Name	Department_ID	Location (Violation)
D501	Dr. Adams	DEP-1	Building A - Floor 3
D503	Dr. Clark	DEP-1	Building A - Floor 3

Table 5: Non-3NF Table: Transitive dependency (*Doctor_ID* $\rightarrow$ *Department_ID* $\rightarrow$ *Location*).

To eliminate the transitive chain and prevent update anomalies (e.g., if a department moves to a new floor), *Location* is completely isolated into the `DEPARTMENT` table.

Department_ID (PK)	Department_Name	Location
DEP-1	Cardiology	Building A - Floor 3
DEP-2	Neurology	Building B - Floor 1

Table 6: 3NF Compliant: Decomposed `DEPARTMENT` relation.

5.4 Boyce-Codd Normal Form (BCNF)

A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

– $\alpha \rightarrow \beta$ is a trivial functional dependency (i.e., $\beta \subseteq \alpha$).

– α is a superkey for schema R.

The schema reaches BCNF, the strictest of the functional dependency-based forms, as the only determinants in our functional dependency set F are superkeys.

- **Proof by Determinant Analysis:** Reviewing the FDs from Section 5:
  - $Patient\_ID \rightarrow \{Name, BirthDate, Gender\}$: Determinant is a Primary Key/Superkey

- $Doctor\_ID \rightarrow \{Name, Specialty, Department\_ID\}$ Determinant is a Primary Key/Superkey
- $Room\_ID \rightarrow \{Room\_Number, Capacity\}$ Determinant is a Primary Key/Superkey

- Since there are no non-trivial functional dependencies $\alpha \rightarrow \beta$ where α is not a superkey, the system is inherently in BCNF. No prime attribute is functionally determined by a non-superkey.

For more information, see Database System Concepts by Silberschatz, Korth and Sudarshan, Chapter 7.3: Normal Forms

6 SQL Implementation

6.1 Data Definition Language (DDL) and Table Constraints

```sql
CREATE DATABASE smarthealth-db:
```

Listing 1: Creating Database

```sql

-- SQL dump generated using DBML (dbml.dbdiagram.io)
-- Database: PostgreSQL

CREATE TABLE "Hospital" (
  "Hospital_ID" int PRIMARY KEY,
  "Hospital_Name" varchar,
  "Address_Country" varchar,
  "Address_City" varchar,
  "Address_District" varchar,
  "Address_Street" varchar,
  "Address_Building_No" varchar,
  "Address_Zip_Code" varchar,
  "Contact_Number" varchar
);

CREATE TABLE "Disease" (
  "Disease_ID" int PRIMARY KEY,
  "Disease_Name" varchar,
  "Disease_Type" varchar
);

CREATE TABLE "Disease_Symptom" (
  "Disease_ID" int REFERENCES "Disease" ("Disease_ID"),
  "Symptom_Name" varchar,
  PRIMARY KEY ("Disease_ID", "Symptom_Name")
);

CREATE TABLE "Insurance_Provider" (
  "Provider_ID" int PRIMARY KEY,
  "Provider_Name" varchar,
  "Tax_Number" varchar,
  "Provider_Email" varchar
);

CREATE TABLE "Person" (
  "Person_ID" int PRIMARY KEY,
  "National_ID" varchar UNIQUE,
  "Name_First_Name" varchar,
```

```
  "Name_Last_Name" varchar,
  "Date_of_Birth" date,
  "Gender" varchar,
  "Blood_Type" varchar,
  "Phone_Number" varchar,
  "Address_Country" varchar,
  "Address_City" varchar,
  "Address_District" varchar,
  "Address_Street" varchar,
  "Address_Building_No" varchar,
  "Address_Apartment_No" varchar,
  "Address_Zip_Code" varchar,
  "Email" varchar UNIQUE,
  "Password_Hash" varchar
);

CREATE TABLE "Department" (
  "Department_ID" int PRIMARY KEY,
  "Hospital_ID" int REFERENCES "Hospital" ("Hospital_ID"),
  "Department_Name" varchar,
  "Extension_Number" varchar
);

CREATE TABLE "Patient" (
  "Person_ID" int PRIMARY KEY REFERENCES "Person" ("Person_ID")
);

CREATE TABLE "Patient_Allergy" (
  "Patient_ID" int REFERENCES "Patient" ("Person_ID"),
  "Allergy_Name" varchar,
  PRIMARY KEY ("Patient_ID", "Allergy_Name")
);

CREATE TABLE "Admin" (
  "Person_ID" int PRIMARY KEY REFERENCES "Person" ("Person_ID"),
  "Role_Level" varchar,
  "MFA_Secret" varchar NOT NULL
);

CREATE TABLE "Staff" (
  "Person_ID" int PRIMARY KEY REFERENCES "Person" ("Person_ID"),
  "Job_Title" varchar
);

CREATE TABLE "Medical_Staff" (
  "Person_ID" int PRIMARY KEY REFERENCES "Person" ("Person_ID"),
  "Department_ID" int REFERENCES "Department" ("Department_ID")
);

CREATE TABLE "Room" (
  "Department_ID" int REFERENCES "Department" ("Department_ID"),
  "Room_Number" varchar,
  "Room_Type" varchar,
  PRIMARY KEY ("Department_ID", "Room_Number")
);

CREATE TABLE "Doctor" (
  "Person_ID" int PRIMARY KEY REFERENCES "Medical_Staff" ("Person_ID"),
```

```
  "Specialty" varchar,
  "License_Number" varchar UNIQUE
);

CREATE TABLE "Doctor_Treatment" (
  "Doctor_ID" int REFERENCES "Doctor" ("Person_ID"),
  "Treatment_Name" varchar,
  PRIMARY KEY ("Doctor_ID", "Treatment_Name")
);

CREATE TABLE "Nurse" (
  "Person_ID" int PRIMARY KEY REFERENCES "Medical_Staff" ("Person_ID")
);

CREATE TABLE "Payer" (
  "Payer_ID" int PRIMARY KEY,
  "Payer_Type" varchar CHECK ("Payer_Type" IN ('Patient', 'Insurance')),
  "Patient_ID" int REFERENCES "Patient" ("Person_ID"),
  "Provider_ID" int REFERENCES "Insurance_Provider" ("Provider_ID"),
  CONSTRAINT check_exclusive_payer CHECK (
    ("Payer_Type" = 'Patient' AND "Patient_ID" IS NOT NULL AND "
       Provider_ID" IS NULL) OR
    ("Payer_Type" = 'Insurance' AND "Provider_ID" IS NOT NULL AND "
       Patient_ID" IS NULL)
  )
);

CREATE TABLE "Insurance" (
  "Patient_ID" int REFERENCES "Patient" ("Person_ID"),
  "Provider_ID" int REFERENCES "Insurance_Provider" ("Provider_ID"),
  "Policy_Number" varchar,
  "Coverage_Rate" decimal CHECK ("Coverage_Rate" >= 0 AND "Coverage_Rate
     " <= 100),
  PRIMARY KEY ("Patient_ID", "Provider_ID")
);

CREATE TABLE "Patient_Disease" (
  "Patient_Person_ID" int REFERENCES "Patient" ("Person_ID"),
  "Disease_Disease_ID" int REFERENCES "Disease" ("Disease_ID"),
  PRIMARY KEY ("Patient_Person_ID", "Disease_Disease_ID")
);

CREATE TABLE "Appointment" (
  "Appointment_ID" int PRIMARY KEY,
  "Patient_ID" int REFERENCES "Patient" ("Person_ID"),
  "Doctor_ID" int REFERENCES "Doctor" ("Person_ID"),
  "Appointment_Date" date,
  "Appointment_Time" time,
  "Status_ID" int CHECK ("Status_ID" IN (0, 1, 2)),
  "Chief_Complaint" varchar
);

CREATE TABLE "Prescription" (
  "Appointment_ID" int PRIMARY KEY REFERENCES "Appointment" ("
     Appointment_ID"),
  "Medication_Name" varchar,
  "Dosage" varchar,
  "Instructions" varchar
```

```
);

CREATE TABLE "Lab_Result" (
  "Lab_ID" int PRIMARY KEY,
  "Appointment_ID" int REFERENCES "Appointment" ("Appointment_ID"),
  "Test_Type" varchar,
  "Result_Value" varchar,
  "Reference_Range" varchar,
  "Test_Date" date
);

CREATE TABLE "Billing" (
  "Bill_ID" int PRIMARY KEY,
  "Appointment_ID" int REFERENCES "Appointment" ("Appointment_ID"),
  "Total_Amount" decimal CHECK ("Total_Amount" >= 0),
  "Patient_Share" decimal CHECK ("Patient_Share" >= 0),
  "Payment_Status" int,
  CONSTRAINT check_share_limit CHECK ("Patient_Share" <= "Total_Amount")
);

CREATE TABLE "Payment" (
  "Payment_ID" int PRIMARY KEY,
  "Bill_ID" int REFERENCES "Billing" ("Bill_ID"),
  "Payer_ID" int REFERENCES "Payer" ("Payer_ID"),
  "Amount_Paid" decimal CHECK ("Amount_Paid" >= 0),
  "Payment_Date" timestamp
);
```

Listing 2: Core Schema Creation and Constraint Definitions

6.2 Data Manipulation Language (DML) and Sample Data Entry

To test the structural correctness of the designed relational database schema, data integrity constraints, and relational flow between tables, a sample dataset was injected into the system. The order in which the data was inserted was designed considering the foreign key dependency chain and topological hierarchy in the database. The '`INSERT`' commands prepared for system testing are presented below:

```


INSERT INTO "Hospital" (
  "Hospital_ID",
  "Hospital_Name",
  "Address_Country",
  "Address_City",
  "Address_District",
  "Address_Street",
  "Address_Building_No",
  "Address_Zip_Code",
  "Contact_Number"
) VALUES
(1, 'Cerrahpasa Faculty of Medicine Hospital', 'Turkey', 'Istanbul', '
    Fatih', 'Kocamustafapasa Ave.', '53', '34098', '+902124143000'),
(2, 'Acibadem Maslak Hospital', 'Turkey', 'Istanbul', 'Sariyer', '
    Buyukdere Ave.', '40', '34457', '+902123044444'),
(3, 'Koc University Hospital', 'Turkey', 'Istanbul', 'Zeytinburnu', '
    Davutpasa Ave.', '4', '34010', '+908502508250'),
(4, 'Hacettepe University Hospital', 'Turkey', 'Ankara', 'Altindag', '
    Sihhiye', '1', '06230', '+903123055000'),
```

```
(5, 'Ege University Faculty of Medicine Hospital', 'Turkey', 'Izmir', 'Bornova', 'Ege University Campus', '1', '35100', '+902323901199'),
(6, 'St Thomas Hospital', 'United Kingdom', 'London', 'Lambeth', 'Westminster Bridge Rd', '1', 'SE1 7EH', '+442071887188'),
(7, 'The Mount Sinai Hospital', 'United States', 'New York', 'Manhattan', 'Gustave L. Levy Pl', '1', '10029', '+12122416500'),
(8, 'Charite - Universitatsmedizin', 'Germany', 'Berlin', 'Mitte', 'Chariteplatz', '1', '10117', '+493045050');

INSERT INTO "Disease" (
  "Disease_ID",
  "Disease_Name",
  "Disease_Type"
) VALUES
(1, 'Type 2 Diabetes', 'Chronic'),
(2, 'Hypertension', 'Cardiovascular'),
(3, 'COVID-19', 'Infectious'),
(4, 'Asthma', 'Chronic'),
(5, 'Rheumatoid Arthritis', 'Autoimmune'),
(6, 'Tuberculosis', 'Infectious'),
(7, 'Alzheimer', 'Neurological'),
(8, 'Influenza (Flu)', 'Infectious'),
(9, 'Leukemia', 'Oncological'),
(10, 'Migraine', 'Neurological');

INSERT INTO "Insurance_Provider" (
  "Provider_ID",
  "Provider_Name",
  "Tax_Number",
  "Provider_Email"
) VALUES
(1, 'Social Security Institution (SGK)', '7720394320', 'contact@sgk.gov.tr'),
(2, 'Allianz Insurance', '0550009620', 'info@allianz.com.tr'),
(3, 'Anadolu Insurance', '0680061327', 'bilgi@anadolusigorta.com.tr'),
(4, 'Bupa Acibadem Insurance', '0070081580', 'info@bupaacibadem.com.tr'),
(5, 'National Health Service (NHS)', 'GB123456789', 'england.contactus@nhs.net'),
(6, 'Medicare', 'US987654321', 'medicare.support@cms.hhs.gov'),
(7, 'Cigna Global', 'GB987654321', 'cignaglobal_customer.care@cigna.com'),
(8, 'Techniker Krankenkasse (TK)', 'DE123456789', 'service@tk.de');

-- Remaining data entries can be found in the sql/data_sample.sql file
```

Listing 3: DML Script

6.3 ALTER Operations

To test the flexibility and evolutionary capacity of the database schema in the live environment, various '`ALTER`' operations were performed. These operations allowed the addition of new business rules and fields to the schema while maintaining the integrity of existing data in the system.

```

ALTER TABLE "Person" ADD COLUMN "Emergency_Contact" varchar;


```

```sql
ALTER TABLE "Appointment" ALTER COLUMN "Status_ID" SET DEFAULT 0;


ALTER TABLE "Billing" ADD CONSTRAINT check_min_bill CHECK ("Total_Amount
    " >= 50) NOT VALID;
```

Listing 4: Database Schema Modification Operations

7 Advanced SQL Queries

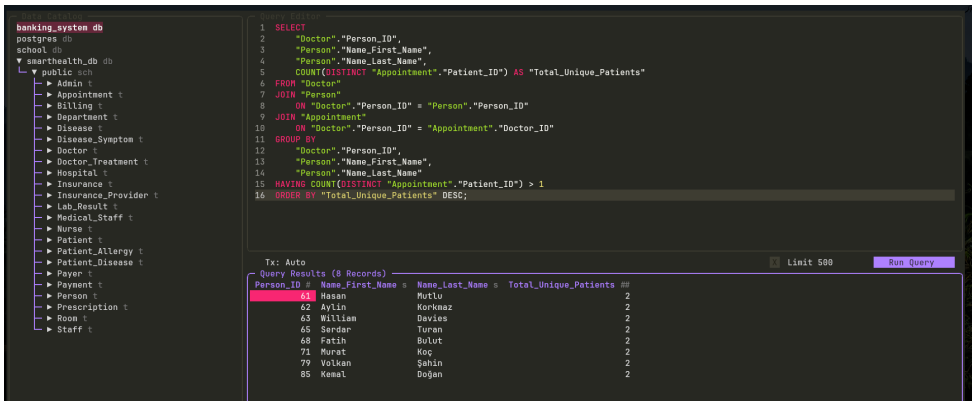

Figure 1: Doctors who see more than one patient

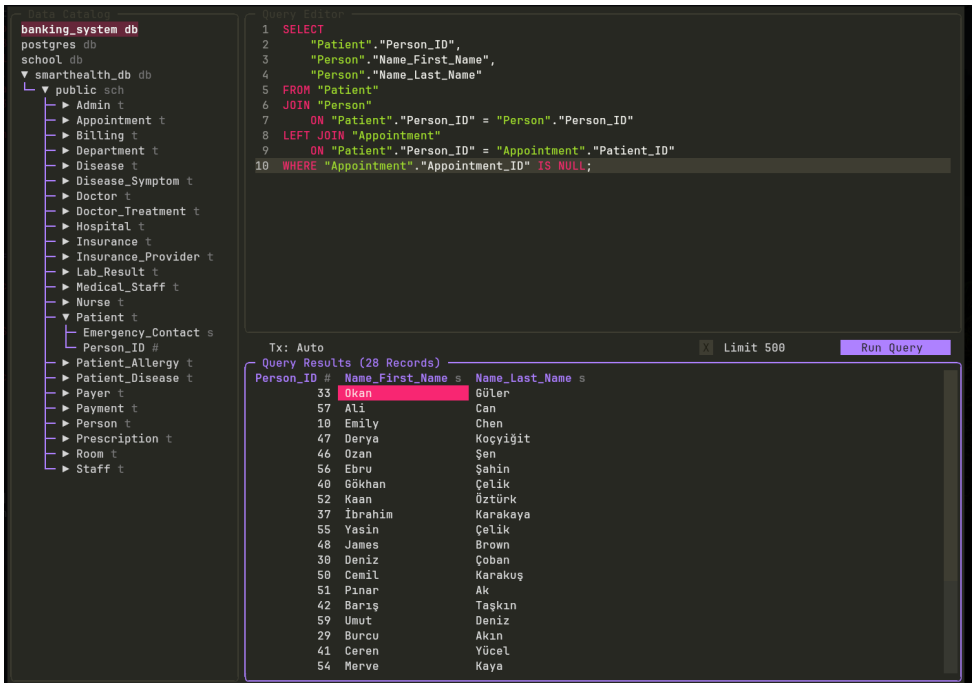

Figure 2: Patients with no appointments

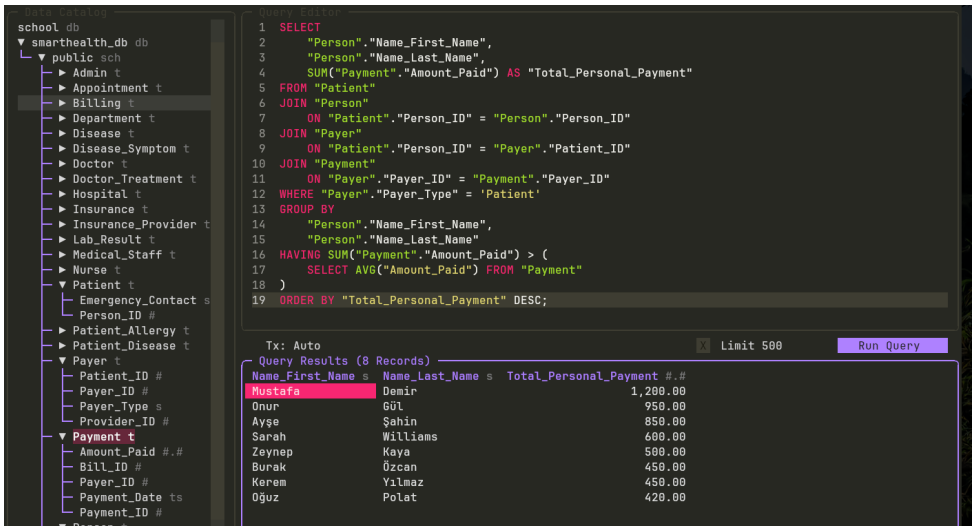

Figure 3: Patients who pay above average

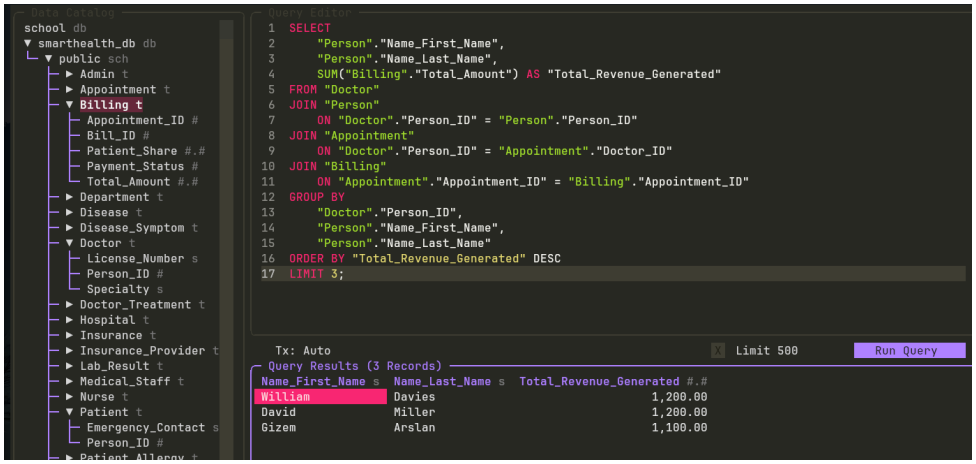

Figure 4: 3 doctors who generate the most revenue for the hospital

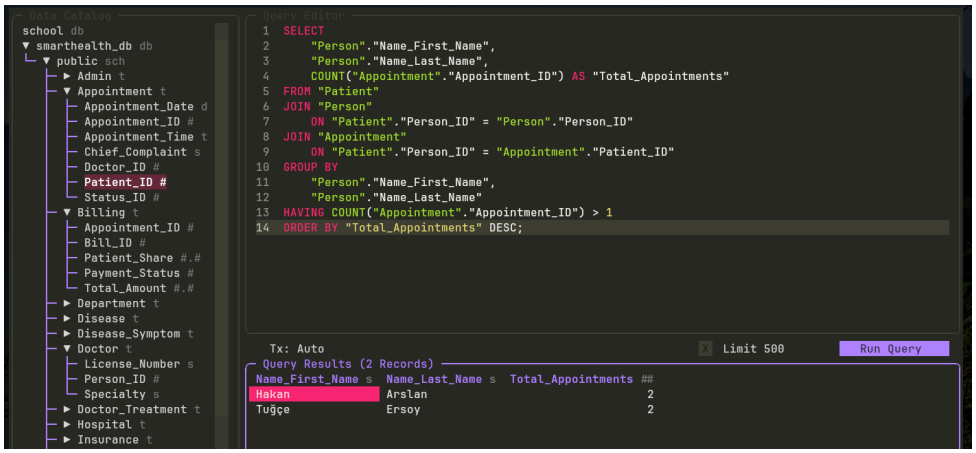

Figure 5: Patients with multiple appointments

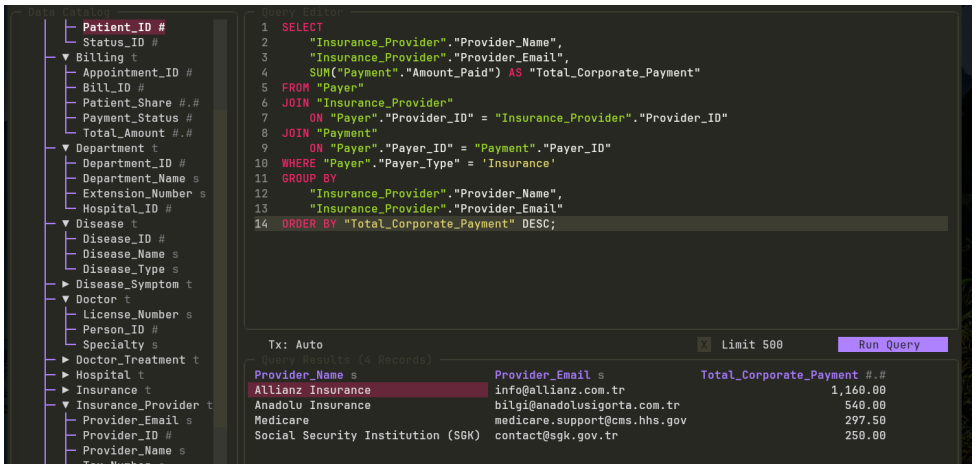

Figure 6: List of insurance providers that make payments

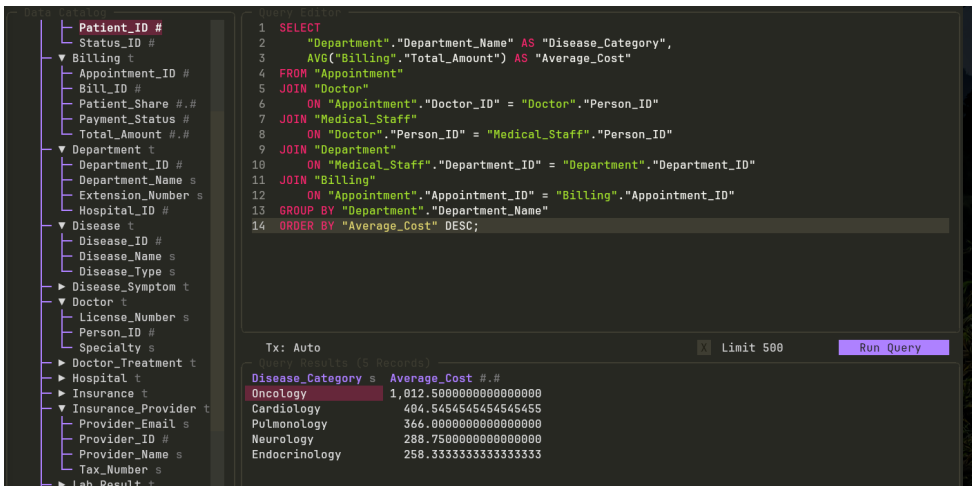

Figure 7: Average cost per appointment for each department

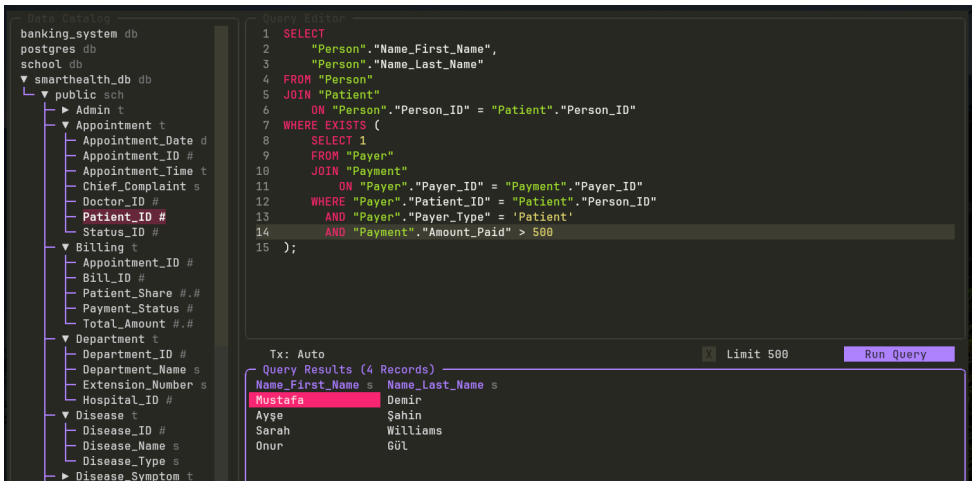

Figure 8: Patients who made a single payment of more than 500

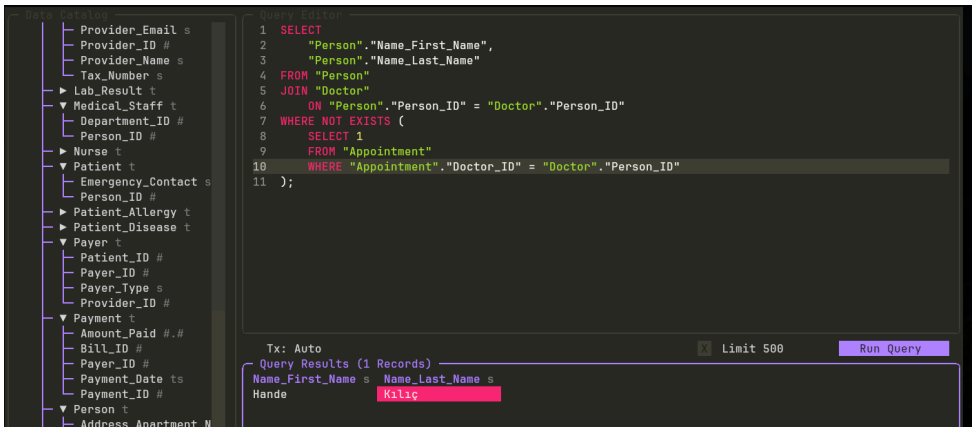

Figure 9: That one doctor who have no appointments

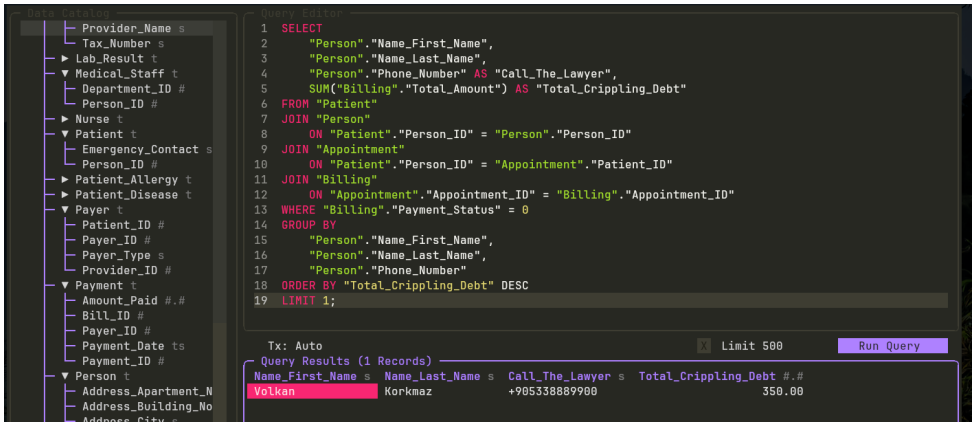

Figure 10: Patient who owes the most money to the hospital and his number to give to the lawyer

8 Relational Algebra & Tuple Relational Calculus

In this section, the abstract mathematical foundations and logical models of the 5 analytical queries executed in our database architecture are expressed in Relational Algebra (RA) and Tuple Relational Calculus (TRC) formats. The γ operator from Extended Relational Algebra was used for grouping and aggregation operations.

1. **Patients with No Appointments**
   List of patients registered in the system who have not yet scheduled an appointment (Set Difference).

```sql
SELECT
    "Patient"."Person_ID",
    "Person"."Name_First_Name",
    "Person"."Name_Last_Name"
FROM "Patient"
JOIN "Person"
    ON "Patient"."Person_ID" = "Person"."Person_ID"
LEFT JOIN "Appointment"
    ON "Patient"."Person_ID" = "Appointment"."Patient_ID"
WHERE "Appointment"."Appointment_ID" IS NULL;
```

Listing 1: Query: Patients with No Appointments

- **Relational Algebra:**

$$\pi_{\text{Person\_ID, Name\_First\_Name, Name\_Last\_Name}}((\text{Patient} \bowtie \text{Person}) \setminus (\text{Patient} \bowtie \text{Person} \bowtie_{\text{Person\_ID=Patient\_ID}} \text{Appointment}))$$

- **Tuple Relational Calculus:**

$$\{t \mid \exists p \in \text{Person}, \exists pat \in \text{Patient} \, (\\ t.\text{Person\_ID} = pat.\text{Person\_ID} \land t.\text{Name\_First\_Name} = p.\text{Name\_First\_Name} \land \\ t.\text{Name\_Last\_Name} = p.\text{Name\_Last\_Name} \land pat.\text{Person\_ID} = p.\text{Person\_ID} \land \\ \neg \exists a \in \text{Appointment} \, (a.\text{Patient\_ID} = pat.\text{Person\_ID}))\}$$

2. **Patients with Multiple Appointments (COUNT > 1)**
Names and total appointment counts of patients who have at least two different appointments.

```sql
SELECT
    "Person"."Name_First_Name",
    "Person"."Name_Last_Name",
    COUNT("Appointment"."Appointment_ID") AS "Total_Appointments"
FROM "Patient"
JOIN "Person"
    ON "Patient"."Person_ID" = "Person"."Person_ID"
JOIN "Appointment"
    ON "Patient"."Person_ID" = "Appointment"."Patient_ID"
GROUP BY
    "Person"."Name_First_Name",
    "Person"."Name_Last_Name"
HAVING COUNT("Appointment"."Appointment_ID") > 1
ORDER BY "Total_Appointments" DESC;
```

Listing 2: Query: Patients with Multiple Appointments

- **Relational Algebra:**

$$\pi_{\text{Name\_First\_Name, Name\_Last\_Name}}(\\ \sigma_{\text{Total\_App}>1}(\text{Name\_First\_Name, Name\_Last\_Name}\gamma_{\text{COUNT(Appointment\_ID)}\to\text{Total\_App}}(\\ \text{Patient} \bowtie \text{Person} \bowtie_{\text{Person\_ID=Patient\_ID}} \text{Appointment})))$$

- **Tuple Relational Calculus (Pure Logical Representation):**

$$\{t \mid \exists p \in \text{Person}, \exists pat \in \text{Patient}, \exists a_1, a_2 \in \text{Appointment} \, (\\ t.\text{Name\_First\_Name} = p.\text{Name\_First\_Name} \land t.\text{Name\_Last\_Name} = p.\text{Name\_Last\_Name} \land \\ pat.\text{Person\_ID} = p.\text{Person\_ID} \land a_1.\text{Patient\_ID} = pat.\text{Person\_ID} \land \\ a_2.\text{Patient\_ID} = pat.\text{Person\_ID} \land a_1.\text{Appointment\_ID} \neq a_2.\text{Appointment\_ID})\}$$

3. **Corporate Clients (Insurances) Leaderboard**
Calculation of total payment amounts generated from insurance-type payers.

```sql
SELECT
    "Insurance_Provider"."Provider_Name",
    "Insurance_Provider"."Provider_Email",
    SUM("Payment"."Amount_Paid") AS "Total_Corporate_Payment"
FROM "Payer"
JOIN "Insurance_Provider"
    ON "Payer"."Provider_ID" = "Insurance_Provider"."Provider_ID"
JOIN "Payment"
    ON "Payer"."Payer_ID" = "Payment"."Payer_ID"
WHERE "Payer"."Payer_Type" = 'Insurance'
```

```
GROUP BY
    "Insurance_Provider"."Provider_Name",
    "Insurance_Provider"."Provider_Email"
ORDER BY "Total_Corporate_Payment" DESC;
```

Listing 3: Query: Corporate Clients Leaderboard

- **Relational Algebra:**

$$_{\text{Provider\_Name, Provider\_Email}}\gamma_{\text{SUM(Amount\_Paid)}\rightarrow\text{Total\_Corp\_Pay}}(\sigma_{\text{Payer\_Type='Insurance'}}(\text{Payer}) \bowtie \text{Insurance\_Provider} \bowtie \text{Payment})$$

- **Tuple Relational Calculus:**

$$\{t \mid \exists ip \in \text{Insurance\_Provider}\,(t.\text{Provider\_Name} = ip.\text{Provider\_Name} \wedge t.\text{Provider\_Email} = ip.\text{Provider\_Email} \wedge t.\text{Total\_Corp\_Pay} = \text{SUM}(\{p.\text{Amount\_Paid} \mid \exists py \in \text{Payer}\,(p.\text{Payer\_ID} = py.\text{Payer\_ID} \wedge py.\text{Provider\_ID} = ip.\text{Provider\_ID} \wedge py.\text{Payer\_Type} = \text{'Insurance'})\}))\}$$

4. **Average Billing Amounts by Department**
Analysis of average revenue generated by medical departments through appointments.

```
SELECT
    "Department"."Department_Name" AS "Disease_Category",
    AVG("Billing"."Total_Amount") AS "Average_Cost"
FROM "Appointment"
JOIN "Doctor"
    ON "Appointment"."Doctor_ID" = "Doctor"."Person_ID"
JOIN "Medical_Staff"
    ON "Doctor"."Person_ID" = "Medical_Staff"."Person_ID"
JOIN "Department"
    ON "Medical_Staff"."Department_ID" = "Department"."
       Department_ID"
JOIN "Billing"
    ON "Appointment"."Appointment_ID" = "Billing"."Appointment_ID"
GROUP BY "Department"."Department_Name"
ORDER BY "Average_Cost" DESC;
```

Listing 4: Query: Average Billing Amounts by Department

- **Relational Algebra:**

$$_{\text{Department\_Name}}\gamma_{\text{AVG(Total\_Amount)}\rightarrow\text{Average\_Cost}}(\text{Appointment} \bowtie_{\text{Doctor\_ID=Person\_ID}} \text{Doctor} \bowtie \text{Medical\_Staff} \bowtie \text{Department} \bowtie \text{Billing})$$

- **Tuple Relational Calculus:**

$$\{t \mid \exists dep \in \text{Department}\,(t.\text{Disease\_Category} = dep.\text{Department\_Name} \wedge t.\text{Average\_Cost} = \text{AVG}(\{b.\text{Total\_Amount} \mid \exists a \in \text{Appointment}, \exists doc \in \text{Doctor}, \exists ms \in \text{Medical\_Staff}\,(b.\text{Appointment\_ID} = a.\text{Appointment\_ID} \wedge a.\text{Doctor\_ID} = doc.\text{Person\_ID} \wedge doc.\text{Person\_ID} = ms.\text{Person\_ID} \wedge ms.\text{Department\_ID} = dep.\text{Department\_ID})\}))\}$$

5. **Phantom Doctors (Doctors with No Appointments)**
Doctors who are registered in the hospital system but have no appointments assigned to them.

```sql
SELECT
    "Person"."Name_First_Name",
    "Person"."Name_Last_Name"
FROM "Person"
JOIN "Doctor"
    ON "Person"."Person_ID" = "Doctor"."Person_ID"
WHERE NOT EXISTS (
    SELECT 1
    FROM "Appointment"
    WHERE "Appointment"."Doctor_ID" = "Doctor"."Person_ID"
);
```

Listing 5: Query: Doctors with No Appointments

- **Relational Algebra:**

$$\pi_{\text{Name\_First\_Name, Name\_Last\_Name}}((\text{Person} \bowtie \text{Doctor}) \setminus (\text{Person} \bowtie \text{Doctor} \bowtie_{\text{Doctor\_ID=Doctor\_ID}} \text{Appointment}))$$

- **Tuple Relational Calculus:**

$$\{t \mid \exists p \in \text{Person}, \exists d \in \text{Doctor}\,(t.\text{Name\_First\_Name} = p.\text{Name\_First\_Name} \wedge t.\text{Name\_Last\_Name} = p.\text{Name\_Last\_Name} \wedge p.\text{Person\_ID} = d.\text{Person\_ID} \wedge \neg\exists a \in \text{Appointment}\,(a.\text{Doctor\_ID} = d.\text{Person\_ID}))\}$$